

Τεχνητή Νοημοσύνη – Εργασία 2

Αλγόριθμοι Τοπικής Αναζήτησης

ΑΡΙΣΤΕΙΔΗΣ ΠΙΤΣΟΥΛΗΣ – 2022030024

ΒΑΣΙΛΕΙΟΣ ΚΑΡΑΒΑΝΑΣ – 2021030132

1 Εισαγωγή

Στην παρούσα εργασία υλοποιούμε τρεις αλγορίθμους τοπικής αναζήτησης με στόχο την εύρεση της καλύτερης ακολουθίας κινήσεων σε δύο γνωστά προβλήματα ελέγχου: *CartPole* και *MountainCar*. Η βασική δυσκολία σε σχέση με κλασικές προσεγγίσεις είναι ότι τα περιβάλλοντα είναι **μη-στατικά** (non-stationary): οι φυσικές τους παράμετροι αλλάζουν δυναμικά κατά την εκτέλεση, οπότε δεν μπορούμε να στηριχτούμε σε ένα σταθερό μοντέλο μετάβασης.

Για την προσομοίωση αυτών των περιβαλλόντων χρησιμοποιούμε τη βιβλιοθήκη `ns-gym`, η οποία επεκτείνει το `gymnasium` ώστε να επιτρέπει μεταβολές παραμέτρων κατά τη διάρκεια του επεισοδίου. Πιο συγκεκριμένα:

- **CartPole**: η μάζα της ράβδου (`masspole`) αυξάνεται σταδιακά μέσω `IncrementUpdate`, ενώ η βαρύτητα (`gravity`) μεταβάλλεται τυχαία μέσω `RandomWalk`.
- **MountainCar**: τόσο η βαρύτητα (`gravity`) όσο και η δύναμη του κινητήρα (`force`) αυξάνονται σταδιακά μέσω δύο ανεξάρτητων `IncrementUpdate`.

Αυτές οι αλλαγές σημαίνουν πρακτικά ότι η συμπεριφορά του περιβάλλοντος διαφέρει από επεισόδιο σε επεισόδιο – ακόμα και εντός του ίδιου επεισοδίου. Αυτό καθιστά αναγκαία τη χρήση online μεθόδων, όπου ο πράκτορας λαμβάνει αποφάσεις βλέποντας μόνο την τρέχουσα κατάσταση.

Οι τρεις αλγόριθμοι που υλοποιούμε – Hill Climbing, Random Restart Hill Climbing και Simulated Annealing – βασίζονται στην ιδέα της τοπικής αναζήτησης όπως παρουσιάζεται στο κεφάλαιο 4 του Russell & Norvig.

2 Αλγόριθμοι Τοπικής Αναζήτησης

2.1 Hill Climbing

Ο αλγόριθμος Hill Climbing (HC) επιλέγει σε κάθε βήμα την κίνηση που οδηγεί στη γειτονική κατάσταση με τη μεγαλύτερη τιμή. Δεν κρατά ιστορικό και δεν κοιτά πέρα από τον επόμενο χρονικό βήμα, οπότε είναι greedy.

Στο πλαίσιο της εργασίας, ο πράκτορας σε κάθε χρονική στιγμή λαμβάνει ένα αντίγραφο του περιβάλλοντος (*planning env*) και αξιολογεί κάθε δυνατή κίνηση προσομοιώνοντας ένα βήμα. Επιλέγεται η κίνηση που δίνει τη μεγαλύτερη τιμή σύμφωνα με τη συνάρτηση `calculate_value`. Αν όλες οι κινήσεις οδηγούν σε τερματισμό, επιστρέφεται -1 και ο πράκτορας επιλέγει τυχαία.

Listing 1: Hill Climbing – `act(env)`

```
1 best_action = -1, best_value = -INF
2 for a in {0, ..., |A|-1}:
3     sim = deepcopy(env)
4     _, r, term, trunc = sim.step(a)
5     if term: continue
6     v = calculate_value(t=sim.env.t, r, term, trunc)
7     if v > best_value:
8         best_value = v; best_action = a
9 return best_action
```

Ο HC έχει το γνωστό μειονέκτημα ότι κολλάει σε τοπικά βέλτιστα. Στο πλαίσιο μας, αυτό σημαίνει ότι μπορεί να επιλέγει πάντα την ίδια κίνηση ακόμα και όταν το περιβάλλον έχει αλλάξει αρκετά.

2.2 Random Restart Hill Climbing

Ο Random Restart Hill Climbing (RRHC) προσπαθεί να ξεπεράσει το πρόβλημα των τοπικών βέλτιστων εκκινώντας πολλαπλές τυχαίες δοκιμές και κρατώντας το καλύτερο αποτέλεσμα.

Στην υλοποίησή μας, σε κάθε χρονική στιγμή εκτελούνται N τυχαίες δοκιμές (restarts). Κάθε δοκιμή επιλέγει τυχαία μια πρώτη κίνηση a_0 και στη συνέχεια συνεχίζει τυχαία για $H-1$ ακόμα βήματα, αθροίζοντας την αξία των καταστάσεων που επισκέπτεται. Η πρώτη κίνηση που αντιστοιχεί στη μεγαλύτερη αθροιστική αξία επιστρέφεται. Χρησιμοποιούμε $N = 20$ restarts και ορίζοντα $H = 5$ βήματα.

Listing 2: Random Restart HC – `act(env)`

```
1 action_values[a] = -INF for each a
2 for i = 1 to N:
3     sim = deepcopy(env); a0 = random(A)
```

```

4     _, r, term, trunc = sim.step(a0)
5     if term: continue
6     total = calculate_value(sim.env.t, r, term, trunc)
7     for j = 2 to H:
8         if term or trunc: break
9         a = random(A)
10        _, r, term, trunc = sim.step(a)
11        total += calculate_value(sim.env.t, r, term, trunc)
12    if total > action_values[a0]:
13        action_values[a0] = total
14    return argmax_a action_values[a]

```

Η χρήση πολλαπλών τυχαίων δοκιμών βοηθά τον πράκτορα να «ξεφύγει» από καταστάσεις όπου ο HC θα έκανε την ίδια επιλογή επανειλημμένα. Αυτό είναι ιδιαίτερα χρήσιμο σε μη-στατικά περιβάλλοντα όπου η βέλτιστη κίνηση αλλάζει με την πάροδο του χρόνου.

2.3 Simulated Annealing

Ο Simulated Annealing (SA) εισάγει μια πιθανοτική αποδοχή χειρότερων κινήσεων, εμπνευσμένη από τη διαδικασία ανόπτησης μετάλλων. Η κεντρική ιδέα είναι ότι στην αρχή (υψηλή θερμοκρασία T) ο πράκτορας εξερευνά ελεύθερα, ενώ σταδιακά (καθώς το T μειώνεται) γίνεται πιο greedy.

Σε κάθε βήμα αξιολογούμε όλες τις διαθέσιμες κινήσεις, βρίσκουμε την καλύτερη a^* και επιλέγουμε τυχαία έναν υποψήφιο a_c . Αν a_c είναι καλύτερος αποδεκτός. Αν είναι χειρότερος, γίνεται αποδεκτός με πιθανότητα $e^{\Delta/T}$, όπου $\Delta = V(a_c) - V(a^*) < 0$.

Listing 3: Simulated Annealing – act(env)

```

1  T = get_temperature();  step_count += 1
2  action_values = {}
3  for a in {0, ..., |A|-1}:
4      sim = deepcopy(env)
5      _, r, term, trunc = sim.step(a)
6      if not term:
7          action_values[a] = calculate_value(sim.env.t, r, term,
            trunc)
8  if action_values is empty: return -1
9  a_best = argmax action_values
10 a_c     = random(action_values.keys())
11 delta   = action_values[a_c] - action_values[a_best]
12 if delta >= 0 or U(0,1) < exp(delta / T):
13     return a_c
14 return a_best

```

Στρατηγικές ψύξης. Πειραματιστήκαμε με τρεις διαφορετικές στρατηγικές μείωσης της θερμοκρασίας, όπου n είναι ο συνολικός αριθμός βημάτων που έχουν εκτελεστεί (δεν επαναφέρεται μεταξύ επεισοδίων):

- **Εκθετική (επιλεγμένη):** $T_n = T_0 \cdot \alpha^n$, με $T_0 = 2.0$, $\alpha = 0.95$.
- **Γραμμική:** $T_n = \max(T_0 - \alpha \cdot n, \varepsilon)$, όπου $\varepsilon = 10^{-3}$.
- **Λογαριθμική:** $T_n = T_0 / \ln(n + 2)$.

Η εκθετική ψύξη είναι η πιο ευρέως χρησιμοποιούμενη στην πράξη γιατί μειώνει ομαλά τη θερμοκρασία και δίνει στον αλγόριθμο αρκετό χρόνο εξερεύνησης πριν συγκλίνει. Η λογαριθμική είναι η πιο αργή και θεωρητικά εγγυάται σύγκλιση στο ολικό βέλτιστο υπό ορισμένες συνθήκες, αλλά στην πράξη με περιορισμένα βήματα δεν αποδίδει απαραίτητα καλύτερα.

3 Υλοποίηση

3.1 Δομή κώδικα

Ο κώδικας οργανώνεται σε δύο φακέλους: `agents/` και `test/`. Στο αρχείο `agents/local_search_agents` ορίζεται η αφηρημένη κλάση `LocalSearchAgent` που παρέχει τις βασικές λειτουργίες, και στο `test/run_experiments.py` βρίσκεται ο κώδικας για τα πειράματα και τα γραφήματα.

3.2 Συνάρτηση αξίας

Η κλάση βάσης ορίζει τη μέθοδο `calculate_value` που υπολογίζει πόσο «καλή» είναι μια κατάσταση:

- **CartPole:** $V = r + 0.1 \cdot \mathcal{K}[\text{trunc}] + t \cdot \mathcal{K}[\neg \text{term}]$

Θέλουμε ο πράκτορας να επιβιώσει όσο γίνεται περισσότερο, οπότε ανταμείβουμε κάθε βήμα που δεν τερματίζει (ο όρος t).

- **MountainCar:** $V = r + 0.1 \cdot \mathcal{K}[\text{trunc}] - t \cdot \mathcal{K}[\neg \text{term}]$

Εδώ θέλουμε ο πράκτορας να φτάσει στον στόχο όσο πιο γρήγορα γίνεται, οπότε το t αφαιρείται.

3.3 Μηχανισμός lookahead

Κάθε φορά που καλείται η `act()`, λαμβάνουμε ένα αντίγραφο του τρέχοντος περιβάλλοντος μέσω `get_planning_env()` και το τυλίγουμε με τον κατάλληλο `wrapper` που τροποποιεί τη συνάρτηση ανταμοιβής. Για κάθε υποψήφια κίνηση δημιουργούμε ένα `deepcopy` αυτού του

αντιγράφου και εκτελούμε ένα βήμα προσομοίωσης, χωρίς να τροποποιούμε το πραγματικό περιβάλλον.

Ένα τεχνικό σημείο: στην έκδοση `gymnasium 1.2.3` που χρησιμοποιούμε, η κλάση `gym.Wrapper` δεν προωθεί αυτόματα την πρόσβαση σε γνωρίσματα στο εσωτερικό περιβάλλον μέσω `__getattr__`. Για αυτό, για να διαβάσουμε τον μετρητή χρόνου `t` χρησιμοποιούμε `sim.env.t` αντί για `sim.t`.

3.4 Simulated Annealing – παρακολούθηση θερμοκρασίας

Ο SA χρειάζεται να ξέρει πόσα βήματα έχουν εκτελεστεί για να υπολογίσει τη θερμοκρασία. Για αυτό, κρατάμε τον μετρητή `step_count` στο αντικείμενο του πράκτορα. Αυτός αυξάνεται σε κάθε κλήση της `act()` και **δεν επαναφέρεται** μεταξύ επεισοδίων – η θερμοκρασία δηλαδή μειώνεται συνεχώς κατά την εκτέλεση (σε όλα τα επεισόδια συνδυαστικά). Αυτή είναι μια σκόπιμη επιλογή: ο αλγόριθμος εξερευνά ελεύθερα στα πρώτα επεισόδια και γίνεται πιο greedy καθώς «μαθαίνει» το περιβάλλον.

4 Πειραματική Διαδικασία

Ακολουθούμε την πειραματική διαδικασία που ορίζει η εκφώνηση. Για κάθε περιβάλλον και κάθε αλγόριθμο:

Πείραμα 1. Τρέχουμε 100 επεισόδια για κάθε τιμή του `max_timesteps` $\in \{10, 100, 500, 1000\}$. Σε κάθε επεισόδιο αθροίζουμε τις ανταμοιβές όπως αυτές επιστρέφονται από το περιβάλλον και υπολογίζουμε τη μέση τιμή από τα 100 επεισόδια. Το αποτέλεσμα απεικονίζεται σε γράφημα με άξονα x το `max_timesteps` και άξονα y τη μέση αθροιστική ανταμοιβή.

Πείραμα 2. Για `max_timesteps = 100` τρέχουμε 1000 επεισόδια και απεικονίζουμε την αθροιστική ανταμοιβή ανά επεισόδιο, ώστε να φαίνεται πώς αλλάζει η συμπεριφορά του πράκτορα καθώς το περιβάλλον μεταβάλλεται.

Η αρχικοποίηση γίνεται με `seed = 42 + ep` για κάθε επεισόδιο `ep`, εξασφαλίζοντας αναπαραγωγιμότητα. Τα πειράματα τρέχουν χωρίς οπτικοποίηση (`render_mode=None`) για ταχύτερη εκτέλεση. Ο κώδικας εκτελείται από το αρχείο `test/run_experiments.py`.

5 Αποτελέσματα

5.1 CartPole

Figure 1: CartPole – Μέση αθροιστική ανταμοιβή συναρτήσει του `max_timesteps` (100 επεισόδια ανά τιμή). Και οι τρεις αλγόριθμοι επιδεικνύουν παρόμοια αύξηση της ανταμοιβής καθώς αυξάνονται τα επιτρεπόμενα βήματα.

Στο Σχήμα 1 παρατηρούμε ότι και οι τρεις αλγόριθμοι αποδίδουν παρόμοια στο CartPole. Η μέση αθροιστική ανταμοιβή αυξάνεται με το `max_timesteps`, κάτι που είναι αναμενόμενο: αφού η ανταμοιβή είναι +1 ανά βήμα επιβίωσης, περισσότερα διαθέσιμα βήματα σημαίνουν δυνητικά μεγαλύτερη αθροιστική ανταμοιβή. Ο HC ανταγωνίζεται αποτελεσματικά τους υπόλοιπους αλγορίθμους, αποδεικνύοντας ότι η greedy ενός βήματος είναι επαρκής για αντιδραστικές αποφάσεις εξισορρόπησης. Ο SA εμφανίζει ελαφρώς μεγαλύτερη διακύμανση λόγω της στοχαστικής αποδοχής, ενώ ο RRHC προσφέρει σταθερή απόδοση μέσω της πολλαπλής εξερεύνησης.

Figure 2: CartPole – Αθροιστική ανταμοιβή ανά επεισόδιο (`max_timesteps`=100, 1000 επεισόδια). Καμία συστηματική υποβάθμιση δεν παρατηρείται παρά τη συνεχή μεταβολή των παραμέτρων.

Στο Σχήμα 2 απεικονίζονται οι ανταμοιβές ανά επεισόδιο για 1000 επεισόδια. Η απόδοση παραμένει σε υψηλά επίπεδα καθ' όλη τη διάρκεια, χωρίς συστηματική μείωση παρά τη συνεχή αύξηση της `masspole` και την τυχαία μεταβολή της `gravity`. Αυτό υποδηλώνει ότι η one-step lookahead προσέγγιση είναι αρκετά ανθεκτική στη μη-στασιμότητα για το CartPole: το άμεσο σήμα ανταμοιβής παρέχει επαρκή πληροφορία για σωστές αποφάσεις ανεξαρτήτως παραμέτρων.

5.2 MountainCar

Figure 3: MountainCar – Μέση αθροιστική ανταμοιβή συναρτήσει του `max_timesteps` (100 επεισόδια). Και οι τρεις αλγόριθμοι αποτυγχάνουν να φτάσουν στον στόχο στην πλειονότητα των επεισοδίων – η ανταμοιβή πλησιάζει τη γραμμή $y = -\text{max_timesteps}$.

Στο Σχήμα 3 παρατηρούμε ότι και οι τρεις αλγόριθμοι σχεδόν δεν διαχωρίζονται – η μέση ανταμοιβή ισούται κατά προσέγγιση με $-\text{max_timesteps}$, πράγμα που σημαίνει ότι ο πράκτορας δεν φτάνει ποτέ στον στόχο και αθροίζει -1 ανά βήμα. Αυτό αποκαλύπτει το βασικό αδύνατο σημείο της one-step lookahead προσέγγισης για το MountainCar: το πρόβλημα απαιτεί συντονισμένες πολλαπλές κινήσεις (συσσώρευση ορμής με εναλλαγή φορά) τις οποίες δεν μπορεί να ανακαλύψει μια στρατηγική που κοιτά μόνο ένα βήμα μπροστά.

Figure 4: MountainCar – Αθροιστική ανταμοιβή ανά επεισόδιο (`max_timesteps=100, 1000` επεισόδια). Σχεδόν όλα τα επεισόδια τερματίζουν με -100 . Σπάνιες εξαιρέσεις δείχνουν κερδισμένα επεισόδια.

Στο Σχήμα 4 διακρίνεται η στενά συγκεντρωμένη ανταμοιβή ≈ -100 για σχεδόν όλα τα επεισόδια. Εμφανίζονται σποραδικά αιχμές (κυρίως από RRHC και SA), που αντιστοιχούν σε επεισόδια όπου η τυχαία εξερεύνηση βοήθησε τον πράκτορα να φτάσει κοντά ή και στον στόχο. Παρατηρείται επίσης μια ελαφρά τάση βελτίωσης στον SA στα μεταγενέστερα επεισόδια, γεγονός που ίσως οφείλεται στη μείωση της θερμοκρασίας – ο αλγόριθμος γίνεται πιο greedy και μπορεί να αξιοποιεί καλύτερα την αυξημένη δύναμη του κινητήρα (`force`) που συσσωρεύεται μέσω `IncrementUpdate`.

6 Συμπεράσματα

Υλοποιήσαμε τρεις αλγορίθμους τοπικής αναζήτησης – HC, RRHC και SA – για online λήψη αποφάσεων σε μη-στατικά περιβάλλοντα. Και οι τρεις χρησιμοποιούν ένα βήμα προσομοίωσης (`lookahead`) για να αξιολογήσουν κάθε υποψήφια κίνηση χωρίς να «ξέρουν» τη δυναμική του συστήματος.

Τα κύρια συμπεράσματα:

- **CartPole (αντιδραστική εργασία):** Και οι τρεις αλγόριθμοι αποδίδουν παρόμοια καλά. Ο HC είναι ανταγωνιστικός, επειδή η εξισορρόπηση ράβδου είναι αντιδραστική εργασία: η greedy επιλογή της καλύτερης άμεσης κίνησης είναι επαρκής. Ο RRHC (ορίζοντας $H = 5, 20$ επανεκκινήσεις) προσφέρει ελαφρώς μεγαλύτερη σταθερότητα σε βάρος υπολογιστικού κόστους. Ο SA δεν παρουσιάζει σαφές πλεονέκτημα εδώ: η στοχαστική αποδοχή χειρότερων κινήσεων δεν βοηθά σε περιβάλλον όπου τα τοπικά βέλτιστα δεν αποτελούν σημαντικό πρόβλημα.
- **Ανθεκτικότητα στη μη-στασιμότητα (CartPole):** Παρά τη συνεχή αύξηση της `masspole` και την τυχαία μεταβολή της `gravity`, δεν παρατηρείται συστηματική υποβάθμιση της απόδοσης κατά τη διάρκεια των 1000 επεισοδίων. Αυτό υποδηλώνει ότι το άμεσο σήμα ανταμοιβής είναι αρκετά πλούσιο ώστε να οδηγεί σε σωστές αποφάσεις ανεξαρτήτως των τιμών των παραμέτρων.
- **MountainCar (εργασία μακράς χρονικής ορατότητας):** Και οι τρεις αλγόριθμοι αποτυγχάνουν σχεδόν πλήρως. Το πρόβλημα απαιτεί συντονισμένη κίνηση σε πολλά βήματα (συσσώρευση ορμής), κάτι που η one-step lookahead δεν μπορεί να αξιολογήσει. Το άμεσο σήμα ανταμοιβής (-1 σε κάθε βήμα) δεν παρέχει χρήσιμη πληροφορία διαφοροποίησης μεταξύ κινήσεων.

- **Προτάσεις βελτίωσης:** Για το MountainCar, ένας μεγαλύτερος ορίζοντας look-ahead (π.χ. $H \gg 5$ στον RRHC) ή πιο ενημερωτική διαμόρφωση ανταμοιβής (π.χ. ανταμοιβή βάσει θέσης) θα βελτίωναν σημαντικά την απόδοση. Εναλλακτικά, προσεγγίσεις model-based που μαθαίνουν τη δυναμική σταδιακά θα ήταν πιο κατάλληλες για τη φύση του προβλήματος αυτού.